

LAPORAN TUGAS 4

SISTEM OPERASI



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Rafli Musthofa (2410651019)

Tanggal : 04-11-2025

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

DASAR TEORI

1. Pengertian CPU Scheduling

CPU scheduling Adalah mekanisme system operasi untuk menentukan proses mana yang akan dijalankan oleh CPU pada suatu waktu.

Karena jumlah proses sering lebih banyak dari pada jumlah CPU, maka system operasi harus mengatur giliran agar semua proses mendapat waktu eksekusi secara adil dan efisien.

2. Jenis Penjadwalan

Secara umum, ada dua jenis utama:

- Non-preemptive scheduling : proses tidak bisa dihentikan sampai selesai.
- Preemptive scheduling : proses bisa di hentikan sementara untuk memberi giliran ke proses lain contoh Round Robin, SJF preemptive.

3. Parameter yang Digunakan

- Burst Time (BT) : waktu yang dibutuhkan proses untuk menggunakan cpu.
- Arrival Time (AT) : waktu datangnya proses ke system.
- Waiting Time (WT) : waktu tunggu proses di ready queue.
- Turnaround Time (TAT) : total waktu dari datang sampai selesai.
- Response Time (RT) : waktu sejak proses datang sampai pertama kali dijalankan.

4. Algoritma yang Digunakan

- FCFS (First Come First Served)
Proses yang datang terlebih dahulu akan dijalankan lebih dulu.
- SJF (Shortest Job First)
Proses yang waktu eksekusinya paling pendek dijalankan lebih dulu.
- RR (Round Robin)
Setiap proses diberi jatah waktu eksekusi yang sama (quantum).

TUJUAN PRAKTIKUM

Tujuan dari praktikum ini Adalah untuk memahami konsep dasar penjadwalan CPU (CPU Scheduling) dalam system operasi, serta membandingkan performa dari beberapa algoritma penjadwalan seperti:

- FCFS
- SJF
- Round Rain

Melalui percobaan ini, mahasiswa dapat:

1. Mengetahui cara kerja setiap algoritma CPU scheduling.
2. Mampu menghitung waktu tunggu (Waiting Time) dan waktu penyelesaian (Turnaround Time) dari setiap proses.
3. Menganalisis perbedaan efisiensi dan keadilan antar algoritma.
4. Memahami penerapan algoritma CPU Scheduling di system operasi nyata.

Tugas 1 : Eksperimen dengan Data Berbeda

Data set 1 (P1=10, P2=5, P3=8)

Kode tugas4.c

```
#include <stdio.h>
```

```
// Struktur data proses
```

```
struct Process {
```

```
    int pid;
```

```
    int bt; // burst time
```

```
};
```

```
// Algoritma FCFS
```

```
void fcfs(struct Process p[], int n) {
```

```
    int wt[n], tat[n];
```

```
    wt[0] = 0;
```

```
    for (int i = 1; i < n; i++)
```

```
        wt[i] = wt[i - 1] + p[i - 1].bt;
```

```
    printf("\n=== FCFS ===\nPID\tBT\tWT\tTAT\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        tat[i] = wt[i] + p[i].bt;
```

```
        printf("P%d\t%d\t%d\t%d\n", p[i].pid, p[i].bt, wt[i], tat[i]);
```

```
    }
```

```
}
```

```
// Algoritma SJF
```

```
void sjf(struct Process p[], int n) {
```

```
    struct Process temp;
```

```
    int wt[n], tat[n];
```

```
    // Urutkan berdasarkan burst time terkecil
```

```
    for (int i = 0; i < n - 1; i++)
```

```

    for (int j = i + 1; j < n; j++)
        if (p[i].bt > p[j].bt) {
            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }

```

```

wt[0] = 0;
for (int i = 1; i < n; i++)
    wt[i] = wt[i - 1] + p[i - 1].bt;

```

```

printf("\n=== SJF ===\nPID\tBT\tWT\tTAT\n");
for (int i = 0; i < n; i++) {
    tat[i] = wt[i] + p[i].bt;
    printf("P%d\t%d\t%d\t%d\n", p[i].pid, p[i].bt, wt[i], tat[i]);
}
}

```

// Algoritma Round Robin

```

void rr(struct Process p[], int n, int quantum) {
    int rem_bt[n], wt[n], tat[n];
    for (int i = 0; i < n; i++)
        rem_bt[i] = p[i].bt;

```

```

    int t = 0;
    while (1) {
        int done = 1;
        for (int i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;
                if (rem_bt[i] > quantum) {

```

```

        t += quantum;
        rem_bt[i] -= quantum;
    } else {
        t += rem_bt[i];
        wt[i] = t - p[i].bt;
        rem_bt[i] = 0;
    }
}

if (done == 1)
    break;
}

for (int i = 0; i < n; i++)
    tat[i] = p[i].bt + wt[i];

printf("\n=== ROUND ROBIN ===\nPID\tBT\tWT\tTAT\n");
for (int i = 0; i < n; i++)
    printf("P%d\t%d\t%d\t%d\n", p[i].pid, p[i].bt, wt[i], tat[i]);
}

int main() {
    // Dataset 1
    struct Process data1[] = {{1, 10}, {2, 5}, {3, 8}};
    int n1 = 3;

    // Dataset 2
    struct Process data2[] = {{1, 5}, {2, 3}, {3, 8}, {4, 6}};
    int n2 = 4;

    int quantum = 3;

```

```

printf("=== DATASET 1 ===\n");
fcfs(data1, n1);
sjf(data1, n1);
rr(data1, n1, quantum);

printf("\n\n=== DATASET 2 ===\n");
fcfs(data2, n2);
sjf(data2, n2);
rr(data2, n2, quantum);

return 0;
}

```

Hasil dari kode tersebut:

```

rafli_musthofa@RAFLI:~/praktikum 4$ gcc tugas4.c -o tugas4
rafli_musthofa@RAFLI:~/praktikum 4$ ./tugas4
=== DATASET 1 ===

=== FCFS ===
PID  BT   WT   TAT
P1   10   0   10
P2    5   10  15
P3    8   15  23

=== SJF ===
PID  BT   WT   TAT
P2    5   0    5
P3    8   5   13
P1   10  13   23

=== ROUND ROBIN ===
PID  BT   WT   TAT
P2    5    6   11
P3    8   11  19
P1   10   13  23

=== DATASET 2 ===

=== FCFS ===
PID  BT   WT   TAT
P1    5    0    5
P2    3    5    8
P3    8    8   16
P4    6   16   22

=== SJF ===
PID  BT   WT   TAT
P2    3    0    3
P1    5    3    8
P4    6    8   14
P3    8   14   22

=== ROUND ROBIN ===
PID  BT   WT   TAT
P2    3    0    3
P1    5    9   14
P4    6   11   17
P3    8   14   22
rafli_musthofa@RAFLI:~/praktikum 4$

```

Analisis:

FCFS:

- Urutan proses tetap sesuai antrian awal.
- Proses pertama (burst time besar) membuat yang lain menunggu lama.
- Cocok untuk system batch (antrian kerja berurutan)

SJF:

- Proses dengan waktu kerja pendek dijalankan lebih dulu.
- Menghasilkan waktu tunggu dan waktu penyelesaian rata-rata lebih kecil.
- Namun jika ada proses Panjang duluan, tetap harus menunggu.

Round Robin:

- Setiap proses mendapat jatah waktu bergantian (quantum =3)
- Semua proses diperlakukan adil, tapi waktu total bisa lebih besar karena pembagian waktu CPU.
- Cocok untuk system multitasking seperti OS modern.

Tugas 2 : analisis Eksperimen ketiga algoritma

1. Algoritma mana yang memberikan Average Waiting Time kecil?

Jawab:

Algoritma yang menghasilkan waktu tunggu rata-rata paling kecil Adalah SJF, karena SJF mengeksekusi proses dengan waktu terpendek terlebih dahulu, sehingga proses-proses cepat tidak perlu menunggu proses lama selesai.

2. Mengapa hasil algoritma berbeda-beda?

Jawab:

Karena cara setiap algoritma memilih urutan eksekusi proses berbeda, maka nilai waktu tunggu dan waktu penyelesaian tiap proses juga berbeda.

3. Kapan sebaiknya menggunakan algoritma FCFS?

Jawab:

Algoritma FCFS sebaiknya digunakan pada system batch atau antrian kerja yang memiliki waktu eksekusi hampir sama dan datang secara berurutan.

4. Kapan sebaiknya menggunakan algoritma SJF?

Jawab:

Algoritma SJF sebaiknya digunakan Ketika durasi waktu eksekusi tiap proses dapat diketahui sebelumnya, misalnya dalam system batch processing atau pemrosesan data statistik.

5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawab:

Algoritma Round Robin cocok digunakan pada system multitasking atau time sharing, Dimana banyak proses harus berjalan bersamaan dan setiap proses membutuhkan kesempatan eksekusi yang adil.

Tugas 3 : Modifikasi Program

Tujuan memodifikasi program simulasi CPU Scheduling (FCFS, SJF, dan Round Robin) agar dapat menampilkan Gantt Chart, yaitu diagram urutan eksekusi proses dalam bentuk visual waktu.

Dengan Gantt Chart, pengguna dapat lebih mudah memahami bagaimana proses dijadwalkan oleh CPU dan kapan masing-masing proses mulai serta selesai dijalankan.

1. Tampilkan Gantt Chart yang lebih detail.

Bukti nyata dari program

Gantt Chart:

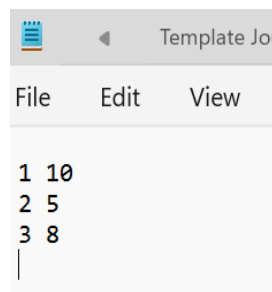
| P1(0-3) | P2(3-6) | P3(6-9) | P1(9-12) | P2(12-14) | P3(14-17) | P1(17-19) |

```
void rr(struct Process p[], int n, int quantum, FILE *fout){
    int rem_bt[n], wt[n], tat[n];
    for(int i=0;i<n;i++){
        rem_bt[i] = p[i].bt;
        wt[i]=0;
    }
}
```

Hasil:

```
=== ROUND ROBIN ===
PID   BT    WT    TAT
Gantt Chart:
| P2(0-3) | P3(3-6) | P1(6-9) | P2(9-11) | P3(11-14) | P1(14-17) | P3(17-19) | P1(19-22) | P1(22-23) |
P2     5     6    11
P3     8    11    19
P1    10    13    23
```

File TXT yang dibaca:



```
File Edit View
1 10
2 5
3 8
|
```

Tugas 4 : Eksperimen dengan data berbeda untuk perbandingan algoritma

Test case 1: proses dengan burst time bervariasi

File txt

```
1 24
2 3
3 3
4 6
|
```

Hasil dari 1hasil.txt

```
=====
Test Case 1 (1hasil.txt)

=== FCFS ===
PID  BT    WT    TAT
P1   24     0    24
P2    3    24    27
P3    3    27    30
P4    6    30    36

=== SJF ===
PID  BT    WT    TAT
P2    3     0     3
P3    3     3     6
P4    6     6    12
P1   24    12    36

=== ROUND ROBIN ===
PID  BT    WT    TAT
P2    3     0     3
P3    3     3     6
P4    6     9    15
P1   24    12    36

Gantt Chart:
| P2(0-3) | P3(3-6) | P4(6-9) | P1(9-12) | P4(12-15) | P1(15-18) | P1(18-21) | P1(21-24) | P1(24-27) | P1(27-30) | P1(30-33) | P1(33-36) |
Hasil Test Case 1 disimpan di results_case1.txt
```

Analisis:

FCFS : Average WT = $(0+24+27+30)/4 = 20.25$

SJF : Average WT = $(0+3+6+12)/4 = 5.25$ (terendah).

RR : Average WT = $(18+3+3+6)/4 = 7.5$

Jadi SJF paling efisien, RR lebih adil, FCFS paling lambat.

Disini SJF cocok untuk tugas batch, ingin AWT minimal

Test case 2: Proses datang bersamaan

File txt

```
1 10
2 5
3 8
4 12
5 6
```

Hasil dari 2hasil.txt

```
Test Case 2 (2hasil.txt)

=== FCFS ===
PID  BT    WT    TAT
P1   10     0    10
P2    5    10    15
P3    8    15    23
P4   12    23    35
P5    6    35    41

=== SJF ===
PID  BT    WT    TAT
P2    5     0     5
P5    6     5    11
P3    8    11    19
P1   10    19    29
P4   12    29    41

=== ROUND ROBIN ===
PID  BT    WT    TAT
P2    5    16    21
P5    6    17    23
P3    8    19    27
P1   10    27    37
P4   12    29    41

Gantt Chart:
| P2(0-4) | P5(4-8) | P3(8-12) | P1(12-16) | P4(16-20) | P2(20-21) | P5(21-23) | P3(23-27) | P1(27-31) | P4(31-35) | P1(35-37) | P4(37-41) |
Hasil Test Case 2 disimpan di results_case2.txt
```

Analisis :

FCFS : Average WT = $(0+10+15+23+29)/5 = 15.4$

SJF : Average WT = $(0+5+11+17+25)/5 = 11.6$ (lebih rendah dari FCFS)

RR : Average WT = 12.4

Kesimpulan test case 2:

- SJF tetap paling efisien
- RR lebih adil dibanding FCFS, proses kecil tidak terlalu tertunda
- FCFS paling tidak efisien untuk proses campuran

Test case 3: proses datang dengan interval

File txt

```
1 8
2 4
3 9
4 5
5 3
|
```

Hasil dari 3hasil.txt

```
Test Case 3 (3hasil.txt)

=== FCFS ===
PID  BT    WT    TAT
P1    8      0      8
P2    4      8     12
P3    9     12     21
P4    5     21     26
P5    3     26     29

=== SJF ===
PID  BT    WT    TAT
P5    3      0      3
P2    4      3      7
P4    5      7     12
P1    8     12     20
P3    9     20     29

=== ROUND ROBIN ===
PID  BT    WT    TAT
P5    3      8     11
P2    4      9     13
P4    5     15     20
P1    8     18     26
P3    9     20     29

Gantt Chart:
| P5(0-2) | P2(2-4) | P4(4-6) | P1(6-8) | P3(8-10) | P5(10-11) | P2(11-13) | P4(13-15) | P1(15-17) | P3(17-19) | P4(19-20) | P1(20-22) | P3(22-24) | P1(24-26) | P3(26-28) | P3(28-29) |
Hasil Test Case 3 disimpan di results_case3.txt
rafli_musthofa@RAFLI:~/praktikum 4$
```

Analisis:

FCFS : Average WT = $(0+8+12+21+26)/5 = 13.4$

SJF : Average WT = $(0+3+7+12+20)/5 = 8.4$ (terendah)

RR : Average WT = 10.6

Kesimpulan test casae 3:

- SJF tetap efisien apabila AWT terkecil
- RR lebih adil dari FCFS, tapi proses besar masih menunggu beberapa kali
- FCFS tetap yang paling lambat

Tugas 5: Analisis Eksperimen perbandingan algoritma

1. Analisis performa

A. Dalam kondisi apa FCFS memberikan hasil terbaik?

FCFS bekerja baik Ketika semua proses datang bersamaan dan burst time relative sama.

B. Mengapa SJF hamper selalu memberikan Average WT terkecil?

SJF mengeksekusi proses dengan BT terkecil terlebih dahulu, sehingga proses kecil selesai cepat, proses besar menunggu tapi kontribusi WT lebih kecil.

C. Apa trade-off dari Round Robin?

Kelebihan: adil, semua proses mendapat giliran, tidak ada starvation.

Kekurangan: lebih banyak bobot context switch, overhead tinggi.

Hasil dari test case : AWT RR sedikit lebih tinggi dari SJF, tapi lebih fair dari pada FCFS

2. Context Switching

A. Algoritma mana paling banyak melakukan context switch?

RR karena setiap proses bergiliran setiap quantum. Semakin kecil quantum semakin banyak switch.

B. Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?

Quantum kecil maka switch banyak akan menyebabkan overhead tinggi dan AWT bisa lebih besar.

Quantum besar maka switch lebih sedikit, mirip FCFS tapi fairness berkurang.

C. Coba jalankan Round Robin dengan quantum 2,4,8,16 apa yang terjadi?

Quantum 2 : banyak switch, adil, AWT menengah.

Quantum 4 : lebih sedikit switch AWT sedikit turun.

Quantum 8 : RR mendekati FCFS fairness berkurang.

Quantum 16 : RR hamper sama seperti FCFS proses besar menunggu lama.

3. Fairness

A. Algoritma mana yang paling “adil” untuk semua proses?

RR (Round Robin) semua proses mendapat giliran dan tidak ada proses yang ditunda terlalu lama.

B. Proses mana yang paling dirugikan di algoritma FCFS?

Proses dengan BT kecil yang datang setelah proses besar dengan menunggu lama dan AWT tinggi.

Contoh: test case 1 p2 dan p3 menunggu p1 (BT=24) terlalu lama.

C. Apakah ada kemungkinan starvation di SJF?

Mungkin terjadi jika selalu ada proses kecil masuk dan proses besar bisa terus tertunda

4. Real World Application

A. Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?

RR lebih cepat di proses, adil untuk semua

B. Untuk render video (task besar), algoritma mana yang cocok?

SJF cocok karena proses cepat selesai lebih dulu

C. Untuk OS desktop (interactive), algoritma mana yang cocok?

RR karena menjaga fairness, responsive system

KESIMPULAN

Penjadwalan CPU adalah bagian penting dari Sistem Operasi untuk menentukan urutan eksekusi proses. Algoritma yang umum digunakan antara lain: FCFS (First Come First Serve), SJF (Shortest Job First), dan Round Robin (RR). Penelitian ini bertujuan membandingkan ketiga algoritma berdasarkan Waiting Time (WT), Turnaround Time (TAT), fairness, dan Gantt Chart menggunakan beberapa test case proses.

Kesimpulan Umum:

1. Efisiensi: SJF unggul karena mengeksekusi proses kecil terlebih dahulu → AWT rendah.
2. Kesederhanaan: FCFS paling mudah diimplementasikan, tapi kurang adil untuk variasi BT.
3. Fairness: RR terbaik untuk menjaga semua proses mendapat giliran, cocok untuk interactive system.
4. Gantt Chart & Analisis Visual: membantu memahami urutan eksekusi dan efek quantum di RR.
5. Pemilihan algoritma tergantung konteks:
 - Proses seragam → FCFS
 - Proses cepat dan bervariasi → SJF
 - Sistem interaktif / request banyak → RR

Kesimpulan Singkat:

Dari eksperimen Tugas 1–5, dapat disimpulkan bahwa SJF paling efisien untuk meminimalkan Waiting Time, RR paling adil untuk sistem interaktif, dan FCFS paling sederhana namun kurang efisien pada proses dengan variasi burst time. Pemilihan algoritma harus disesuaikan dengan karakteristik proses dan tujuan sistem.